

دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

درس برنامه سازی وب

نیم سال بهار ۱۴۰۵
استاد درس: علی ابریشمی

گزارش نهایی پروژه درس

SoundWave یک سرویس استریم موسیقی

نوشته شده توسط:

فواد خیرآبادی - ۴۰۲۱۰۵۹۷۶
ایلیا فرصتی - ۴۰۲۱۰۶۲۹۷
رایان حدیدی - ۴۰۲۱۱۱۶۴

فهرست مطالب

۱	مقدمه و شرح کلی پروژه	۲
۲	نقش و کارهای هر عضو در هر فاز	۲
۱.۲	فاز اول - فرانت‌اند	۲
۲.۲	فاز دوم - بک‌اند	۳
۳.۲	تست‌نویسی	۳
۳	قوانین توسعه و فرآیند کار	۳
۱.۳	شاخه‌بندی گیت	۳
۲.۳	قالب پیام کامیت	۳
۳.۳	قراردادهای نام‌گذاری	۴
۴.۳	فرآیند ادغام فرانت‌اند و بک‌اند	۴
۴	ساختار پروژه و توجه آن	۴
۱.۴	ساختار فرانت‌اند	۴
۲.۴	ساختار بک‌اند	۵
۵	توجه نگهداری‌پذیری کد	۵
۶	مدل‌های موجود در بک‌اند، روابط و توجه آن‌ها	۶
۱.۶	توجه روابط کلیدی	۸
۷	نمونه کد تولیدشده با کمک هوش مصنوعی	۸
۱.۷	فاز اول - فرانت‌اند: سیستم پیشنهاددهنده آهنگ	۸
۲.۷	فاز دوم - بک‌اند: لایه انتزاعی درگاه پرداخت	۹
۸	نقاط قوت و ضعف هوش مصنوعی در توسعه فرانت‌اند در برابر بک‌اند	۱۱
۱.۸	فرانت‌اند	۱۱
۲.۸	بک‌اند	۱۱
۹	جمع‌بندی	۱۲

۱ مقدمه و شرح کلی پروژه

SoundWave یک سرویس استریم موسیقی مشابه Spotify است که طی دو فاز درس برنامه‌سازی وب توسعه داده شد: فاز اول با تمرکز بر رابط کاربری و منطق سمت کلاینت (Next.js 14 با TypeScript)، و فاز دوم با تمرکز بر پیاده‌سازی بک‌اند واقعی (Django REST Framework + Django) و ادغام آن با فرانت‌اند موجود.

سامانه چهار نقش کاربری دارد: **شنونده** (کاربر عادی)، **هنرمند** (پس از تأیید هویت می‌تواند اثر منتشر کند)، **پشتیبان** (بررسی درخواست‌های تأیید هنرمند و رسیدگی به تیکت‌ها) و **مدیر سامانه** (دسترسی کامل، شامل قیمت‌گذاری و تسویه حساب هنرمندان). سه سطح اشتراک پایه (رایگان)، نقره‌ای و طلایی محدودیت‌های متفاوتی در تعداد استریم روزانه، تعداد پلی لیست، دانلود آهنگ، دسترسی زودهنگام به انتشارهای جدید و مشاهده آمار هنرمندان اعمال می‌کنند؛ این مقادیر به صورت رکوردهای قابل ویرایش در دیتابیس (نه مقادیر ثابت در کد) نگهداری می‌شوند تا مدیر سامانه بتواند بدون نیاز به دیپلوی مجدد، سیاست قیمت‌گذاری را تغییر دهد.

پروژه توسط سه نفر و به صورت تیمی توسعه داده شد. مالکیت هر بخش از فرانت‌اند در فاز اول، مستقیماً مالکیت اپ‌های Django متناظر آن در فاز دوم را تعیین کرد؛ یعنی هرکس صفحه/کامپوننتی را در فاز اول ساخته بود، در فاز دوم منطق سمت سرور همان بخش را نیز پیاده‌سازی کرد. این تصمیم، آشنایی عمیق هر عضو با دامنه کاری خودش را از فاز اول به فاز دوم منتقل کرد و از هزینه یادگیری مجدد دامنه توسط فرد دیگر جلوگیری کرد.

۲ نقش و کارهای هر عضو در هر فاز

جدول زیر مالکیت کلی هر عضو در دو فاز پروژه را نشان می‌دهد؛ توضیحات دقیق‌تر هر بخش در ادامه آمده و شرح کامل هر مورد در فایل‌های TASKS_FOAD.md، TASKS_ILIYA.md و TASKS_RAYAN.md در ریشه مخزن قابل مشاهده است.

عضو	فاز اول (فرانت‌اند)	فاز دوم (بک‌اند)
فواد	احراز هویت (ورود/ثبت‌نام)، اعلانات، پنل مدیریت هنرمند، داشبورد پشتیبانی/مدیر، PWA	support, billing, accounts, notifications
ایلیا	پخش‌کننده موسیقی (پلیر)، سیستم پیشنهاددهنده آهنگ با هوش مصنوعی	playback, music
رایان	صفحه خانه، نوار کناری، پروفایل کاربر/هنرمند، کتابخانه/جستجو، پلی لیست‌ها، تنظیمات	social, playlists

۱.۲ فاز اول - فرانت‌اند

فواد. صفحه ورود و ثبت‌نام با اعتبارسنجی zod + react-hook-form (تب جداگانه برای شنونده و هنرمند)، هدایت پس از ورود بر اساس نقش کاربر (شنونده/هنرمند → خانه، پشتیبان/مدیر → داشبورد پشتیبانی)، صفحه اعلانات با وضعیت خوانده/نخوانده و حذف/علامت‌گذاری، پنل مدیریت آثار هنرمند (افزودن/ویرایش/حذف اثر همراه با آمار هر اثر)، داشبورد پشتیبانی/مدیر با چهار تب (تأیید هویت هنرمندان، تیکت‌های پشتیبانی، حسابرسی، مدیریت اشتراک‌ها و قیمت‌گذاری)، و پیاده‌سازی PWA (آیکون‌ها، manifest، صفحه آفلاین، دکمه نصب برنامه).

ایلیا. پخش‌کننده موسیقی پرارزش‌ترین بخش فاز اول - شامل نوار پخش ثابت دسکتاپ (کاور، عنوان، هنرمند، آلبوم، کنترل‌های پخش/توقف/بعدی/قبلی، نوار پیشرفت با قابلیت جابه‌جایی، دکمه تکرار سه‌حالت و شافل، کنترل صدا)، حالت میلی‌پلیر و پلیر تمام‌صفحه در موبایل، پنل صف پخش و پنل نمایش متن آهنگ، و یکپارچه‌سازی Howler.js از طریق هوک اختصاصی useHowler. همچنین ویجت پیشنهاددهنده آهنگ مبتنی بر هوش مصنوعی را در صفحه خانه جای‌گذاری و به تاریخچه پخش واقعی کاربر متصل کرد.

رایان. نوار کناری (ناوبری دسکتاپ + نوار پایین/کشوی موبایل)، صفحه خانه (پلی لیست‌های اخیر، آلبوم‌های جدید، پرفردارترین آهنگ‌ها، بخش دسترسی زودهنگام ویژه اشتراک طلایی)، صفحات پروفایل کاربر و هنرمند

(آمار دنبال‌کننده، نشان اشتراک، دکمه دنبال‌کردن)، صفحه تنظیمات، صفحات پلی‌لیست (ایجاد/حذف/تغییر نام با اعمال محدودیت تعداد بر اساس سطح اشتراک) و صفحه کتابخانه/ جستجو (جستجو، مرتب‌سازی، افزودن به پلی‌لیست).

۲.۲ فاز دوم - بک‌اند

فواد. اپ accounts: سریالایزهای ثبت‌نام شنونده/هنرمند، تولید خودکار username، جاسازی نقش کاربر در JWT، فراموشی رمز عبور با پاسخ یکنواخت (برای جلوگیری از افشای وجود ایمیل). اپ billing: پنل قیمت‌گذاری مدیر، اتصال به درگاه پرداخت ZarinPal (verify_payment/request_payment)، فعال‌سازی/تمدید اشتراک پس از تأیید پرداخت، تسویه حساب ماهانه هنرمندان از طریق دستور مدیریتی calculate_payouts، و گزارش‌های تجمیعی درآمد/توزیع اشتراک. اپ support: چت‌باکس تیکت، تأیید/رد درخواست هویت هنرمند. اپ notifications: علامت‌گذاری خواننده/همه‌خواننده و صدور اعلان از رخدادهای اپ‌های دیگر.

ایلیا. اپ music: مجوز ایجاد/ویرایش اثر منحصر به هنرمند تأییدشده مالک آن، اعتبارسنجی و ذخیره فایل صوتی/کاور، محاسبه تعداد استریم و شنوندگان یکتا (نمایش شنوندگان یکتا فقط برای مشترکان طلایی)، اعمال سقف استریم روزانه پیش از ثبت رویداد پخش، جستجو و مرتب‌سازی. اپ playback: همگام‌سازی تنظیمات کاربر بین دستگاه‌ها و اندپوینت «اخیراً پخش‌شده» که ورودی واقعی سیستم پیشنهاددهنده آهنگ را تأمین می‌کند. رایان. اپ playlists: اعمال سقف تعداد پلی‌لیست بر اساس سطح اشتراک، مدیریت افزودن/حذف آهنگ با حفظ ترتیب (position)، محدودسازی دسترسی به پلی‌لیست‌های خود کاربر. اپ social: آمار دنبال‌کننده/دنبال‌شونده، دنبال‌کردن/لغو دنبال‌کردن با جلوگیری از دنبال‌کردن خود و از ثبت رکورد تکراری.

۳.۲ تست‌نویسی

طبق سیاست‌نامه پروژه، فاز اول حداقل ۱۰ تست و فاز دوم حداقل ۱۵ تست جنگو نیاز داشت. هر عضو در هر دو فاز حداقل ۴ تا ۵ تست در حوزه کاری خود نوشت (مثلاً فواد: ثبت‌نام/ورود، دسترسی ادمین به تغییر قیمت، فعال‌سازی اشتراک پس از پرداخت؛ ایلیا: محدودیت استریم روزانه، مجوز هنرمند تأییدشده؛ رایان: سقف پلی‌لیست، عدم امکان دنبال‌کردن خود) تا مجموع حداقل‌های تعیین‌شده در هر دو فاز تأمین شود.

۳ قوانین توسعه و فرآیند کار

۱.۳ شاخه‌بندی گیت

```
main          # stable, always deployable
dev           # integration branch
feature/foad-* # Foad's features
feature/iliya-* # Iliya's features
feature/ryan-* # Rayan's features
```

هر ویژگی در یک شاخه feature/<owner>-* توسعه و از طریق Pull Request به main ادغام می‌شد (مطابق تاریخچه گیت پروژه، بیش از ده PR جداگانه برای بخش‌های مختلف از جمله back-music-playback-enhancements, phase2-feature-pages, end-initialize و support-english مرور و ادغام شده‌اند).

۲.۳ قالب پیام کامیت

از قالب Conventional Commits استفاده شد تا تاریخچه پروژه خوانا و قابل جستجو بماند:

```

feat(auth): add JWT login endpoint
fix(player): correct repeat mode cycling
style(sidebar): align nav icons
refactor(models): simplify subscription logic
test(api): add track CRUD tests
docs(readme): update setup instructions

```

۳.۳ قراردادهای نام‌گذاری

مثال	قرارداد	نوع
TrackCard.tsx	PascalCase	کامپوننت React
usePlayerStore.ts	camelCase با پیشوند use	هوک React
TrackViewSet	PascalCase	View در Django
ArtistProfile	PascalCase	مدل Django
Player.test.tsx	test_*.py یا *.test.tsx	فایل تست

همچنین در کل پروژه هیچ مقدار رنگی به صورت مستقیم در کد فرانت‌اند نوشته نمی‌شود؛ تمام رنگ‌ها، فاصله‌گذاری‌ها و تایپوگرافی از طریق متغیرهای CSS تعریف شده در styles/globals.css (مانند var(--color-primary)) مصرف می‌شوند تا تغییر بعدی سیستم طراحی در یک فایل متمرکز باشد، نه پراکنده در ده‌ها کامپوننت.

۴.۳ فرآیند ادغام فرانت‌اند و بک‌اند

در فاز اول، فرانت‌اند به طور کامل روی توابع mock (مانند mockGetPlaylists و mockGetNotifications) و Local Storage کار می‌کرد. در فاز دوم، لایه lib/api/* (شامل auth.ts، music.ts، billing.ts، playlists.ts و مشابه) نوشته شد تا هر یک از این توابع mock با فراخوانی واقعی به /api/v1/<app>/... جایگزین شود، بدون آنکه ساختار کامپوننت‌ها تغییر اساسی کند. مدل داده‌ای پاسخ بک‌اند از طریق توابع نگاشت (lib/api/mappers.ts) به همان تایپ‌های TypeScript تعریف شده در فاز اول تبدیل می‌شود؛ این تصمیم عمداً اتخاذ شد تا تغییر منبع داده (از mock به API واقعی) کمترین تأثیر را روی کد کامپوننت‌ها بگذارد.

۴ ساختار پروژه و توجیه آن

مخزن به دو پوشه مستقل تقسیم شده که هر کدام قابل اجرای جداگانه‌اند و از طریق docker-compose.yml نیز می‌توانند هم‌زمان اجرا شوند:

```

SoundWave/
|-- soundwave-frontend/ # Next.js 14 (React, TypeScript)
|-- soundwave-backend/ # Django + Django REST Framework (Phase 2)
`-- docker-compose.yml # docker compose up -> runs both services

```

۱.۴ ساختار فرانت‌اند

فرانت‌اند از App Router در Next.js 14 استفاده می‌کند و مسیرها با گروه‌های مسیریابی (auth)، (main) و (dashboard) از هم جدا شده‌اند این جداسازی به هر گروه اجازه می‌دهد layout و منطق نگهداری نقش (RequireRole) اختصاصی خودش را داشته باشد، بدون آنکه صفحات عمومی (ورود/ثبت‌نام) به اشتباه در چیدمان اصلی برنامه (با نوار کناری و پخش‌کننده) رندر شوند.

```

app/
  (auth)/login, register # public pages, no sidebar/player
  (main)/home, library, ... # main app shell, behind login guard
  (dashboard)/support    # behind support/admin role guard
  api/suggest-songs      # route handler for the AI suggester
components/
  auth/, player/, playlist/, dashboard/, ui/, ... # one folder per domain
lib/
  api/      # backend integration layer (phase 2)
  hooks/    # custom hooks (useHowler, useSongSuggestions)
  store/    # Zustand: authStore, playerStore, localeStore
  validators/ # zod schema per form
  mock/     # phase 1 mock data/functions (kept for tests/local dev)

```

کامپوننت‌ها بر اساس دامنه کسب‌وکار (player, playlist, dashboard, ...) دسته‌بندی شده‌اند نه بر اساس نوع فایل؛ این ساختار مستقیماً با تقسیم مالکیت تیم (جدول بخش ۲) هم‌راستا است و باعث می‌شود هر عضو تیم عمدتاً در پوشه خودش کار کند و تداخل merge کاهش یابد. کامپوننت‌های عمومی و بدون منطق دامنه (دکمه، مودال، جدول، ورودی) در components/ui جدا شده‌اند تا در همه صفحات یکسان استفاده شوند.

۲.۴ ساختار بک‌اند

بک‌اند به‌صورت یک اپلیکیشن Django با پوشه apps/ تقسیم‌شده بر اساس دامنه (نه بر اساس نوع فایل) سازمان‌دهی شده است الگوی رایج و توصیه‌شده در Django برای پروژه‌های با چند دامنه مستقل:

```

soundwave-backend/
  soundwave/ # project settings, root urls.py
  apps/
    accounts/ # User, ArtistProfile -> Foad
    billing/  # SubscriptionPlan, Subscription,
              # PaymentTransaction, Payout -> Foad
    support/  # Ticket, TicketMessage -> Foad
    notifications/ # Notification -> Foad
    music/    # Album, Track, StreamEvent -> Iliya
    playback/ # UserPreference -> Iliya
    playlists/ # Playlist, PlaylistTrack -> Rayan
    social/   # Follow -> Rayan

```

این تفکیک هشت‌گانه، دقیقاً هم‌راستا با تقسیم مالکیت فرانت‌اند در فاز اول انتخاب شد (بخش ۲) تا هر عضو بتواند بدون هماهنگی مداوم با دو نفر دیگر، هم مدل و هم منطق API حوزه خودش را مستقل توسعه دهد. هر اپ چهار فایل استاندارد Django دارد (urls.py, views.py, serializers.py, models.py) و همه مسیرها زیر .../api/v1/<app>/ با یک router به‌ازای هر اپ ثبت می‌شوند این یکنواختی باعث می‌شود افزودن اپ جدید در آینده الگوی از پیش شناخته‌شده‌ای داشته باشد.

۵ توجه نگهداری‌پذیری کد

چند تصمیم طراحی مشخصاً با هدف نگهداری‌پذیری (maintainability) گرفته شده‌اند:

- **عدم تکرار منطق دسترسی.** محدودیت‌های سطح اشتراک (سقف پلی لیست، سقف استریم روزانه، دسترسی زود هنگام) همگی در جدول SubscriptionPlan نگهداری می‌شوند، نه به‌صورت عدد ثابت در چند view پراکنده؛ تغییر سیاست قیمت‌گذاری فقط یک رکورد دیتابیس را تغییر می‌دهد، نه کد را.

- **عدم ساخت مدل زائد.** برای مثال، اپ `playback` به‌جای مدلی مجزا برای تاریخچهٔ پخش، از `StreamEvent` موجود در `music` به‌عنوان منبع واحد حقیقت استفاده می‌کند (به‌جای دو مدل که باید همیشه هم‌زمان به‌روزرسانی شوند). به همین ترتیب «تک‌آهنگ» به‌جای مدل مستقل، صرفاً یک `Album` با `release_type=single` و یک `Track` است.
- **تجمیع در بک‌اند، نه فرانت‌اند.** آمار شنونده/استریم/درآمد همیشه در بک‌اند محاسبه و به‌صورت خلاصه (نه رکورد خام) در اختیار فرانت‌اند قرار می‌گیرد؛ این کار هم از نشت دادهٔ خام کاربران جلوگیری می‌کند و هم منطق محاسبه را در یک نقطه نگه می‌دارد.
- **لایهٔ انتزاعی درگاه پرداخت.** `apps/billing/gateways.py` یک کلاس پایهٔ `PaymentGateway` با متدهای `verify_payment/request_payment` تعریف می‌کند؛ `views.py` فقط با این واسط کار می‌کند، نه با `API` مشخص `ZarinPal`. جایگزینی یا افزودن درگاه دوم صرفاً نیازمند یک کلاس جدید در همین فایل است.
- **تایپ‌های مشترک بین فاز اول و دوم.** تایپ‌های `TypeScript` تعریف‌شده در فاز اول (`types/index.ts`) بدون تغییر در فاز دوم نیز استفاده می‌شوند؛ لایهٔ `lib/api/mappers.ts` پاسخ `API` واقعی را به همین تایپ‌ها نگاشت می‌کند تا کامپونت‌ها از منبع داده (`mock` در برابر `API`) بی‌اطلاع بمانند.
- **تست به‌عنوان مستندسازی رفتار.** حداقل تست تعیین‌شده برای هر فاز (۱۰ فرانت‌اند / ۱۵ بک‌اند) باعث شد رفتارهای مرزی حساس مانند «کاربر رایگان پس از رسیدن به سقف استریم مسدود می‌شود» یا «کاربر نمی‌تواند پلی‌لیست دیگری را ویرایش کند» به‌صورت اجراپذیر مستند شوند، نه فقط در کامنت کد.

۶ مدل‌های موجود در بک‌اند، روابط و توجیه آن‌ها

بک‌اند شامل ۱۲ مدل در ۸ اپ است. جدول زیر مدل‌ها، فیلدهای کلیدی و روابط آن‌ها را خلاصه می‌کند.

روابط اصلی	مدل	اپ
کاربر پایه با نقش (listener/artist/support/admin): مبنای همه کلیدهای خارجی کاربر در سایر اپ‌ها	User	accounts
OneToOne با User؛ وضعیت تأیید هویت (pending/approved/rejected)	ArtistProfile	accounts
ForeignKey به User؛ توکن یک‌بار مصرف با انقضای ۱ ساعته	PasswordResetToken	accounts
مستقل؛ تعریف سه سطح اشتراک و محدودیت‌های هر کدام	SubscriptionPlan	billing
ForeignKey به User و SubscriptionPlan؛ دوره فعال/منقضی اشتراک	Subscription	billing
ForeignKey به User؛ ForeignKey اختیاری (nullable) به Subscription (چون اشتراک تا موفقیت پرداخت ساخته نمی‌شود)	PaymentTransaction	billing
ForeignKey به ArtistProfile؛ (artist_profile, period_month) یکتا بر	Payout	billing
ForeignKey به User	Ticket	support
ForeignKey به Ticket و User (فرستنده)	TicketMessage	support
ForeignKey به User؛ نوع اعلان با TextChoices	Notification	notifications
ForeignKey به ArtistProfile؛ نوع انتشار (single/album)	Album	music
ForeignKey به Album؛ ManyToMany با ArtistProfile (همکاران اثر)	Track	music
ForeignKey به Track و User؛ منبع واحد حقیقت برای آمار پخش	StreamEvent	music
OneToOne با User؛ تنظیمات همگام‌شونده بین دستگاه‌ها	UserPreference	playback
ForeignKey به User (مالک)	Playlist	playlists
جدول واسط Track×Playlist با فیلد position برای حفظ ترتیب؛ یکتا بر (track, playlist)	PlaylistTrack	playlists
User خودارجاع بر (followee/follower)؛ CheckConstraint برای جلوگیری از دنبال کردن خود	Follow	social

۱.۶ توجیه روابط کلیدی

- **User** در مرکز گراف مدل قرار دارد: تقریباً همه مدل‌های دیگر مستقیم یا غیرمستقیم (از طریق **ArtistProfile**) به آن ارجاع می‌دهند. این طراحی طبیعی است چون هر چهار نقش سامانه (شنونده، هنرمند، پشتیبان، مدیر) در واقع همان مدل **User** با مقدار متفاوت فیلد **role** هستند نیازی به مدل جداگانه برای هر نقش نبود.
- **ArtistProfile** به جای افزودن فیلد به **User**: به صورت **OneToOneField** جدا نگه داشته شده چون فقط برای کاربران با **role=artist** معنا دارد (وضعیت تأیید، نام هنری، پورتفولیو): افزودن این فیلدها مستقیماً به **User** باعث می‌شد اکثر شنوندگان مقادیر **null** بی‌مصرف داشته باشند.
- **PlaylistTrack** به عنوان جدول واسطه صریح: چون **ManyToManyField** ساده جنگو نمی‌تواند فیلد اضافی مثل **position** (ترتیب آهنگ در پلی‌لیست) را نگه دارد، یک مدل واسطه جداگانه تعریف شد.
- **Follow** خودارجاع: چون هم شنونده می‌تواند شنونده را دنبال کند و هم شنونده هنرمند را (هر دو یک **User** هستند)، یک مدل **Follow** کافی است و نیازی به مدل جداگانه **ArtistFollow** نبود.
- **PaymentTransaction** کلید خارجی اختیاری به **Subscription** دارد: چون یک تراکنش پرداخت ممکن است هرگز موفق نشود و بنابراین هرگز **Subscription** متناظری ساخته نشود؛ اطلاعات سطح/مدت درخواستی موقتاً روی خود تراکنش (**duration_months**, **plan_tier**) نگه داشته می‌شود تا **callback** درگاه پرداخت بتواند بعداً **Subscription** را بسازد.

۷ نمونه کد تولیدشده با کمک هوش مصنوعی

۱.۷ فاز اول - فرانت‌اند: سیستم پیشنهاددهنده آهنگ

فایل `app/api/suggest-songs/route.ts` یک `Route Handler` در `Next.js` است که تاریخچه پخش و ژانرهای موردعلاقه کاربر را می‌گیرد، یک `prompt` فارسی برای یک مدل زبانی می‌سازد و حداکثر سه پیشنهاد آهنگ همراه با دلیل کوتاه برمی‌گرداند. این فایل ابتدا به صورت اسکلت توسط هوش مصنوعی نوشته شد و سپس توسط ایلیا به صفحه خانه متصل و به تاریخچه پخش واقعی (`usePlayerStore`) وصل شد.

```
export async function POST(req: NextRequest) {
  try {
    const body: SuggestionRequest = await req.json();
    const { recentlyPlayedIds, likedGenres, dislikedTrackIds = [], mood } = body;

    // Only what the model needs is sent, never the raw catalog
    const catalog = MOCK_TRACKS
      .filter((t) => !dislikedTrackIds.includes(t.id))
      .map((t) => ({
        id: t.id,
        title: t.title,
        genre: t.genre ?? 'Unknown',
        artists: t.artists.map((a) => a.stageName).join(', '),
        streamCount: t.streamCount,
      }));

    // Prompt body is written in Persian in the actual source file;
    // shown here in English for readability of this report.
    const prompt = `
```

```

You are a music recommendation system for Soundwave.
...
Task: suggest at most 3 tracks from the catalog above that
match the user's taste and mood. Return exactly this JSON
format, with no extra text:
[{ "trackId": "t1", "reason": "short reason in Persian" }]
`.trim();

const response = await fetch(
  'https://api.groq.com/openai/v1/chat/completions',
  {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      Authorization: `Bearer ${process.env.GROQ_API_KEY}`,
    },
    body: JSON.stringify({
      model: 'llama-3.3-70b-versatile',
      max_tokens: 1000,
      messages: [{ role: 'user', content: prompt }],
    }),
  },
);

const data = await response.json();
const clean = (data.choices?.[0]?.message?.content ?? '[]')
  .replace(/```json|```/g, '')
  .trim();
const suggestions: SuggestionResult[] = JSON.parse(clean);

const result = suggestions
  .map((s) => {
    const track = MOCK_TRACKS.find((t) => t.id === s.trackId);
    return track ? { track, reason: s.reason } : null;
  })
  .filter(Boolean);

return NextResponse.json({ suggestions: result });
} catch (err) {
  return NextResponse.json(
    { error: 'error message in Persian' },
    { status: 500 }
  );
}
}

```

نکته: با آنکه در طراحی اولیه سند پروژه این ویژگی «مبتنی بر Claude» توصیف شده بود، پیاده‌سازی نهایی از API مکالمه Groq با مدل Llama 3.3 70B استفاده می‌کند؛ رابط (interface) کد به گونه‌ای طراحی شده که تعویض ارائه‌دهنده مدل زبانی صرفاً به تغییر این یک فایل محدود بماند.

۲.۷ فاز دوم - بک‌اند: لایه انتزاعی درگاه پرداخت

فایل `apps/billing/gateways.py` یک نمونه نماینده کد بک‌اند است که با کمک هوش مصنوعی و بر اساس مستندات `sandbox` درگاه `ZarinPal` (لینک‌شده در سند مشخصات پروژه) نوشته شد. طراحی آن عمده‌ا پشت یک کلاس پایه انتزاعی قرار گرفته تا `views.py` هرگز مستقیماً به جزئیات یک درگاه خاص وابسته نباشد:

```

class PaymentGateway:
    def request_payment(self, *, amount, description, callback_url):
        raise NotImplementedError

    def verify_payment(self, *, reference_id, amount) -> bool:
        raise NotImplementedError

class ZarinPalGateway(PaymentGateway):
    """https://www.zarinpal.com/docs/paymentGateway/sandBox.html"""

    def __init__(self):
        sandbox = settings.PAYMENT_GATEWAY_SANDBOX
        self.start_pay_base = (
            "https://sandbox.zarinpal.com" if sandbox
            else "https://payment.zarinpal.com"
        )
        self.api_base = (
            "https://sandbox.zarinpal.com" if sandbox
            else "https://api.zarinpal.com"
        )
        self.merchant_id = settings.PAYMENT_GATEWAY_MERCHANT_ID

    def request_payment(self, *, amount, description, callback_url):
        response = requests.post(
            f"{self.api_base}/pg/v4/payment/request.json",
            json={
                "merchant_id": self.merchant_id,
                "amount": int(amount),
                "description": description,
                "callback_url": callback_url,
            },
            timeout=10,
        )
        response.raise_for_status()
        data = response.json().get("data") or {}
        if data.get("code") != 100:
            raise PaymentGatewayError(f"rejected: {response.json()}")
        authority = data["authority"]
        return f"{self.start_pay_base}/pg/StartPay/{authority}", authority

    def verify_payment(self, *, reference_id, amount) -> bool:
        response = requests.post(
            f"{self.api_base}/pg/v4/payment/verify.json",
            json={
                "merchant_id": self.merchant_id,
                "amount": int(amount),
                "authority": reference_id,
            },
            timeout=10,
        )
        response.raise_for_status()
        data = response.json().get("data") or {}
        # 100 = verified now, 101 = already verified (duplicate callback)
        return data.get("code") in (100, 101)

```

۸ نقاط قوت و ضعف هوش مصنوعی در توسعه فرانت‌اند در برابر بک‌اند

۱.۸ فرانت‌اند

نقاط قوت:

- تولید سریع کامپوننت‌های تکراری اما با جزئیات زیاد (فرم‌های ثبت‌نام دوتبه، جدول‌های داشبورد، کارت‌های آلبوم/آهنگ) که ساختار مشابه اما فیلدهای متفاوت دارند.
- نوشتن اسکیمای اعتبارسنجی zod مستقیماً از روی توضیحات فیلد در سند مشخصات، با سرعت بسیار بالاتر از نوشتن دستی.
- حفظ سازگاری با سیستم طراحی (استفاده از متغیرهای CSS به جای رنگ ثابت) وقتی الگوی کامپوننت‌های قبلی به آن نشان داده می‌شد.
- تولید تست‌های اولیه Jest برای رفتارهای ساده (رندر شدن، تغییر حالت با کلیک).

نقاط ضعف:

- جزئیات ظریف تعامل کاربر (مثلاً کشیدن نوار پیشرفت پخش‌کننده برای seek، ژست swipe برای بستن پلیر تمام صفحه در موبایل) نیاز به راهنمایی دقیق‌تر و بازبینی بصری توسط انسان داشت کدی که در نگاه اول درست به نظر می‌رسید، در تست واقعی روی مرورگر رفتار نامطلوب داشت.
- هماهنگی وضعیت بین چند فروشگاه Zustand (مثلاً playerStore و authStore) گاهی بدون بازبینی انسانی دچار وابستگی ضمنی یا رندر اضافه می‌شد.
- بررسی واکنش‌گرایی (responsive) در پهنای صفحه موبایل عملاً غیرقابل اعتماد بدون مشاهده مستقیم در مرورگر بود؛ کد از نظر منطقی درست اما از نظر چیدمان بصری نیازمند اصلاح دستی می‌شد.

۲.۸ بک‌اند

نقاط قوت:

- ترجمه سریع و دقیق بخش‌های مشخصات پروژه (که با شماره بند، مثل spec §۴.۲)، در کامنت‌های کد ارجاع داده شده‌اند) به الگوهای استاندارد Django REST Framework (Serializer, ViewSet, Permission سلفارشی).
- رعایت مداوم قید صریح مشخصات پروژه («فقط داده تجمیعی به فرانت‌اند برود، هرگز رکورد خام») در تمام اندپوینت‌های گزارش‌گیری، وقتی این قید به صراحت در دستور کار قرار می‌گرفت.
- تولید مجموعه تست یکنواخت برای هشت اپ مختلف با ساختار مشابه (مجوز دسترسی، محدودیت سطح اشتراک، روابط مالکیت) که نوشتن دستی تکراری آن‌ها زمان‌بر بود.
- طراحی لایه انتزاعی (مانند PaymentGateway) وقتی صراحتاً خواسته می‌شد کد به یک ارائه‌دهنده خاص وابسته نباشد.

نقاط ضعف:

- قوانین کسب‌وکاری که مشخصات پروژه عمداً باز گذاشته بود (مانند فرمول دقیق تسویه حساب ماهانه هنرمندان از total_streams/unique_listeners) نمی‌توانست توسط هوش مصنوعی به‌تنهایی تصمیم‌گیری شود و نیازمند تصمیم صریح تیم بود.

- ملاحظات امنیتی و همزمانی (concurrency) در مسیر بازگشتی درگاه پرداخت مثل برخورد با callback تکراری یا محافظت merchant_id نیازمند بازبینی دقیق انسانی بود؛ کد تولیدی اولیه این حالت‌های مرزی را همیشه پوشش نمی‌داد.
- بدون تأکید صریح، کد پیش‌فرض تولیدشده گاهی به‌جای رعایت کامل اصل «جنگو-محور» (استفاده از ابزارهای DRF به‌جای کد تکراری دستی)، الگوهای عمومی‌تر و کمتر بهینه پیشنهاد می‌داد که نیاز به بازنویسی مطابق قراردادهای DRF داشت.

۹ جمع‌بندی

پروژه SoundWave با تقسیم‌بندی دامنه‌محور ثابت بین فاز اول و دوم، قراردادهای مشخص نام‌گذاری/شاخه‌بندی/کامیت، و استفاده هدفمند از هوش مصنوعی به‌عنوان دستیار (نه تصمیم‌گیرنده نهایی) توسعه یافت. نتیجه نهایی یک سرویس استریم موسیقی کامل با چهار نقش کاربری، سه سطح اشتراک قابل‌مدیریت از پنل مدیر، اتصال واقعی به درگاه پرداخت، و یک ویژگی امتیازی مبتنی بر مدل زبانی بزرگ برای پیشنهاد آهنگ است.